

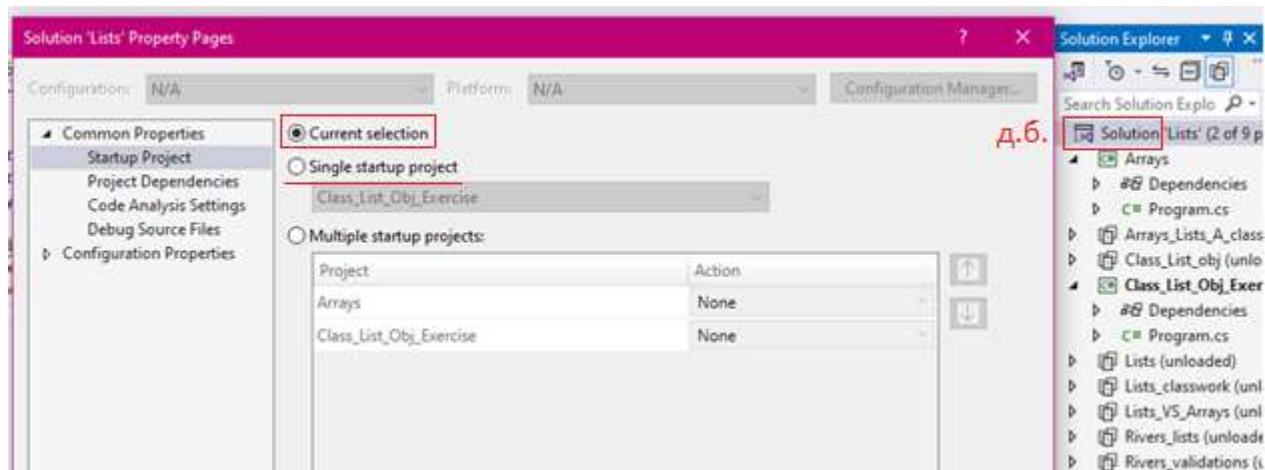
СПИСЪК ОТ КЛАСОВЕ – пояснения и надграждане

Според системните въпроси и грешки, съм подредила няколко пояснения:

1) – Активен проект

До скоро не одобрявах да копирате готов код от предходни задачи, затова ви показвах само метода с Reload/Unload.

Когато в рамките на 1 Solution имаме много проекти (които сме добавяли с д.б. -> Add),



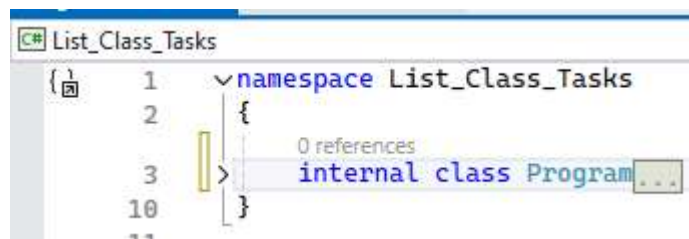
Можем да влезем на малко по-дълбоко ниво във възможностите на VS и с д.б. върху нашия **Solution > Properties -> Current selection** – за текущия селектиран/избран проект.

Така ще може да имаме много заредени и разгърнати проекти в рамките на нашия Solution, техният код да ни бъде достъпен, но активният (с удебелен шрифт **Bold**) ще бъде този, върху който сме кликнули.

По подразбиране е Single startup project (да има само 1 startup проект).

2) – Не преименувайте class Program!!!

Най-добре е да пазим входната точка на програмата - нашият Main() в class Program възможно най-изчистена за по-добра оптимизация на кода и ако трябва, го свиваме, за да разположим новия персонален клас на правилното място според отварящите и затварящи къдрави скоби {}. В следващите часове ще разгледаме случаи, в които може даден клас да бъде вложен в друг, но засега ще подреждаме новите класове **ПОД** Program.



Понеже разгледам какви решения предлага DeepSeek, който е чатбот, по-профилиран в математика и програмиране, ми правят впечатление и няколко особености при него.

- Той е на първо място чат-бот, а не програмист. Знае принципно как трябва да се направи нещо, но не го прави така, както би го направил човек, който разбира, а като чат-бот, който се старае възможно най-бързо да даде работещо решение, макар и не оптимално. Също, не открива собствените си грешки, дори да му бъдат посочени, съгласява се и прави същото (дори синтаксисът на грешката и на корекцията му са 1:1).

Например, той винаги изписва първо класа, после Main(). Защо? – Такава е последователността на решение принципно – винаги първо си построяваме класа, след това го използваме, само че, програмистът поставя класа под Main().

Друг пример е, че DeepSeek системно НЕ използва конструктори, т.е. не предлага оптимално решение.

Трети пример е, че избягва по-новите вградени методи, а използва по-старите, макар, да е запознат. Разбира се, предлага по няколко възможности.

Има още неща, които съм забелязала, че прави, но накратко – необходими са му още много тренировки, за да направи нещата достатъчно качествено. Други чат-ботове, които е хубаво да се разгледат, са ChatGPT, Claude и вграденият Copilot.

Така че, за мен, специално предназначеният учебен материал за даден език като туториълите, остават безценни.

Къде е по-правилно да се разполагат новите класове: под или над class Program

В C# няма строго правило, но има добра практика. Ето препоръките:

 **Препоръчително: Класове ПОД Program**

Защо тази структура е по-добра:

1. По-лесно за четене

csharp

```
// Четеш отгоре-надолу:  
// 1. Първо виждаш основната логика (Program.Main)  
// 2. После виждаш помощните класове
```

2. Подобно на други езици

- Java: main метод е отгоре
- C++: main функция е отгоре
- Python: `if __name__ == "__main__"`

3. Visual Studio подразбиране

Когато създаваш нов проект, Program.cs винаги съдържа Program класа най-отгоре.

Алтернативни структури:

Структура 2: Отделни файлове (Най-добра за големи проекти)

text

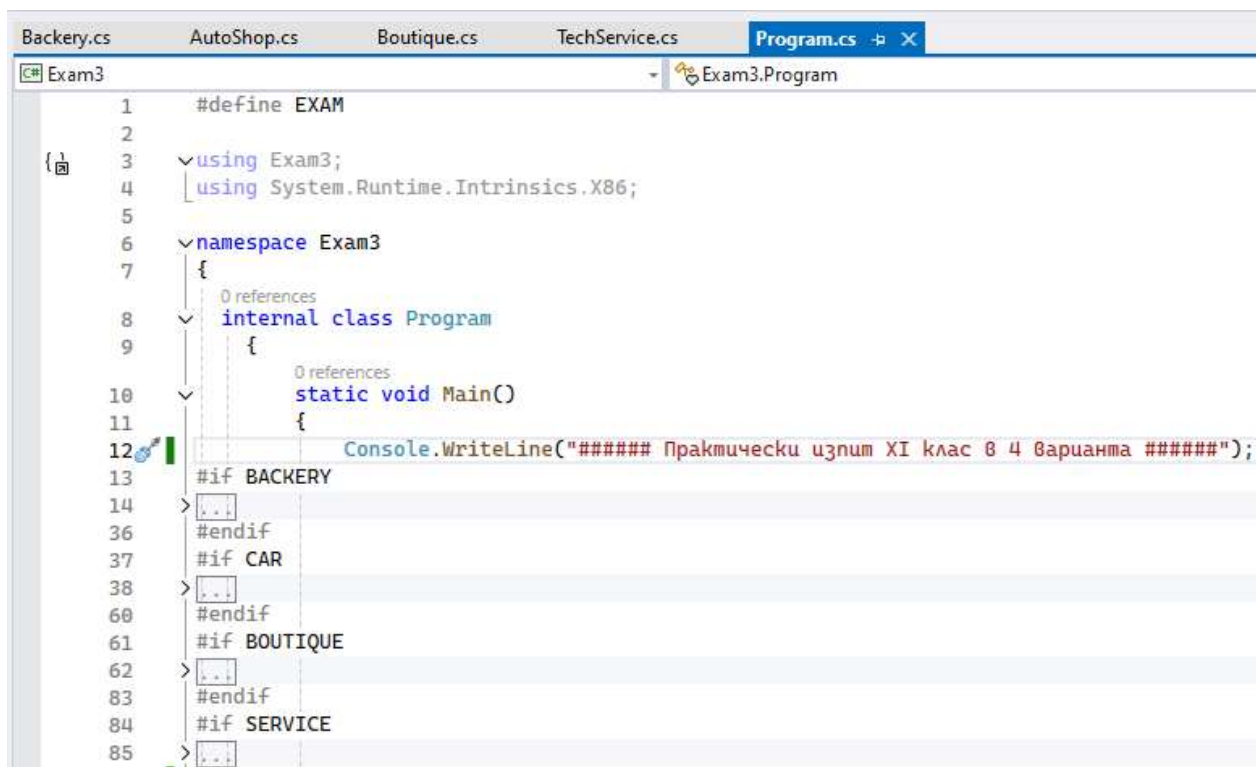
 Copy

```
Project/  
├─ Program.cs  
├─ Car.cs  
└─ Motorcycle.cs
```

Сравнение:

Позиция	Предимства	Недостатъци
Program ОТГОРЕ	✓ Лесно намираш Main ✓ Стандартна практика ✓ Подобно на други езици	✗ Трябва да скролваш за други класове
Program ОТДОЛУ	✓ Виждаш първо дефинициите	✗ Трудно намираш Main ✗ Нестандартно

3) Какво е #if?



```
1  #define EXAM
2
3  using Exam3;
4  using System.Runtime.Intrinsics.X86;
5
6  namespace Exam3
7  {
8      0 references
9      internal class Program
10     {
11         0 references
12         static void Main()
13         {
14             Console.WriteLine("##### Практически узлит XI клас 8 4 варианта #####");
15         }
16     }
17 }
18
19 #if BACKERY
20 > ...
21 #endif
22
23 #if CAR
24 > ...
25 #endif
26
27 #if BOUTIQUE
28 > ...
29 #endif
30
31 #if SERVICE
32 > ...
33 #endif
```

```

#else
    // Ако искаме да стартираме гадена задача, в #define изпишем коя: BACKERY, CAR, BOUTIQUE, SERVICE
    try
    {
        // Тук мествам кода на ученици
        #region
    }
    catch (Exception ex)
    {
        Console.WriteLine("Error: " + ex.Message);
    }

    Console.ReadKey();

#endif
Console.WriteLine("##### XI class #####");
}
// класът на ученика:
7 references
internal class ServiceRepair ...
}

```

Още нещо, което не съм показвала от предходното контролно, че Split() има overload:

```

if (string.IsNullOrEmpty(value) || value.Split(',', StringSplitOptions.RemoveEmptyEntries).Length < 4)
    throw new ArgumentException("Трябва да въведете поне 4 продукта, разделени със запетая!");

```

4) – Какво е това .this

Ние работим с официалната конвенция на Microsoft, която има следната структура. Вътре в конструктора, за да се избегне повторение name = name, се използва this. за private – променливата. Ние използваме най-често Name = name, защото искаме да включим валидациите на set в public – променливата.

<pre> public class Student { private string name; // private none - camelCase private int age; public string Name // public property - PascalCase { get { return name; } set { name = value; } } public Student(string name, int age) { this.name = name; // this за разграничаване this.age = age; } } </pre>	<pre> public class Student { private string name; private int age; private double grade; private List<string> subjects; public string Name => name; public int Age => age; public double Grade => grade; public Student(string name, int age, double grade) { this.name = name; this.age = age; this.grade = grade; this.subjects = new List<string>(); } } </pre>
---	---

При втория синтаксис имаме **read-only properties with backing fields** – много по-голяма защита (но и ограниченост)

```
private string name; // ✅ Напълно private поле
public string Name => name; // ✅ Read-only property
```

В някои проекти се използва конвенция **underScore** – когато има много private-полета без валидации в класа. Чрез тях се избягва `this.`, защото няма повторения.

```
public class Car
{
    private string _model;
    private int _year;

    public string Model
    {
        get { return _model; }
        set { _model = value; }
    }

    public Car(string model, int year)
    {
        _model = model; // Няма нужда от "this"
        _year = year;
    }
}
```

```
public class Person
{
    private string _name; // private field с _
    private int _age;

    public string Name // public property без _
    {
        get => _name;
        set => _name = value;
    }

    public int Age
    {
        get => _age;
        set => _age = value;
    }
}
```

Има дори остаряла конвенция – Унгарска (Hungarian):

```
private string strName; // Остаряла практика
private int iAge; // Не се препоръчва в C#
```

Други възможности:

```
public string Name { get; set; } // ❌ Всеки може да променя
```

```
public string Name { get; private set; } // ✅ Само класът може да променя
```

```

public class Student
{
    public string Name { get; private set; }    //  Защитено
    public int Age { get; private set; }
    public double Grade { get; private set; }

    public Student(string name, int age, double grade)
    {
        //  Единственото място за промяна - с валидация!
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Името не може да е празно");
        if (age < 6 || age > 100)
            throw new ArgumentException("Невалидна възраст");
        if (grade < 2 || grade > 6)
            throw new ArgumentException("Невалидна оценка");

        Name = name;
        Age = age;
        Grade = grade;
    }
}

```

Във версия C# 9+ се въвежда **init**:

```

public class Student
{
    public string Name { get; init; }    // Може да се инициализира само при създаване
    public int Age { get; init; }
    public double Grade { get; init; }

    // Няма нужда от конструктор!
}

// Използване:
Student student = new Student { Name = "Иван", Age = 16, Grade = 5.75 };

```

5) – Как да си подреждаме кода в задачите?

```

internal class Program
{
    0 references
    static void Main()
    {
        List<Employee> employees = EnterEmployees();
        HighestPaid(employees);
    }

    1 reference
    private static void HighestPaid(List<Employee> employees)...

    1 reference      return      Метод()
    private static List<Employee> EnterEmployees()...
    }

    9 references
    public class Employee...

```

- 1) Създаваме класа
- 2) Създаваме логиката, която ни трябва със списъци
- 3) Създаваме метода (който търси, сумира или като цяло изчислява).

Когато сме готови с логиката на нашия работещ вярно код, най-добре е да го отделим извън Main().

```

public class Employee
{
    3 references
    public string Name { get; set; }
    3 references
    public string Position { get; set; }
    5 references
    public decimal Salary { get; set; }

    1 reference
    public Employee(string name, string position, decimal salary)
    {
        Name = name;
        Position = position;
        Salary = salary;
    }

    0 references
    public override string ToString() //стандартен, много инфо при гебъз
    {
        return $"Име: {Name}, Длъжност: {Position}, Заплата: {Salary:F2}";
    }

    1 reference
    public void DisplayInfo()
    {
        Console.WriteLine($"Име: {Name}, Длъжност: {Position}, Заплата: {Salary:F2}");
    }
}

```


Новото в случая е стандартният метод:

```
public override string ToString() //стандартен, много инфо при дебъг
{
    return $"Име: {Name}, Длъжност: {Position}, Заплата: {Salary:F2}";
}
```

Който не отпечатва резултат, а връща текст (return string).

1. `ToString()` е метод от базовия клас `Object`

- Всеки клас в C# наследява от `Object`
- `ToString()` е виртуален метод в `Object`
- Използваме `override`, за да пренапишем съществуващ метод

Създаваме метод, който приема стойности за служители и връща като резултат списък. Най-добре е той да бъде автоматично генериран с `Ctrl + .`, за да сме сигурни, че е поставен на правилното място извън `Main()`. Чрез `static` указваме, че **методът принадлежи на класа** (Program), а не на даден обект.

```
private static List<Employee> EnterEmployees()
{
    return Method();
    List<Employee> employees = new List<Employee>();

    Console.WriteLine("Въведете информация за служителите (за край въведете 'край' за име):");

    while (true)
    {
        // ...
    }

    return employees;
}
```

По-лесният вариант е да знаем предварително броят на служителите, които ще въведем от конзолата. Тогава задачата би станала с `for`. Ако, обаче, решим да използваме по-гъвкав подход, той може да се осъществи чрез `while`:

```

while (true)
{
    Console.Write("Име: ");
    string name = Console.ReadLine();

    if (string.IsNullOrEmpty(name))... валидация

    if (name.ToLower() == "край") Условие за край веднага след въвеждането
        break;

    string position;
    do... валидация

    Console.Write("Заплата: ");
    decimal salary;
    while (!decimal.TryParse(Console.ReadLine(), out salary) || salary < 0)... валидация

    employees.Add(new Employee(name, position, salary));

    Console.WriteLine("Служителят е добавен успешно!\n");
}

```

В случая, веднага след въвеждането на Име се проверява дали не е въведена думата „край“ за стоп. Имаше и 3 валидации, които са разположени непосредствено след въвеждането на име, длъжност и заплата, за да изведат съобщение за грешка непосредствено след въвеждането.

```

while (true)
{
    Console.Write("Име: ");
    string name = Console.ReadLine();

    if (string.IsNullOrEmpty(name))
    {
        Console.WriteLine("Името не може да бъде празно! Опитайте отново.");
        continue;
    }
}

```

Обърнете внимание, че тук не използваме ключовата дума `value`, т.к. не сме в `set`.

```

string position;
do
{
    Console.Write("Длъжност: ");
    position = Console.ReadLine();
    if (string.IsNullOrEmpty(position))
        Console.WriteLine("Длъжността не може да бъде празна! Опитайте отново.");
} while (string.IsNullOrEmpty(position));

```

Виждаме в действие различни конструкции – всяка задача има по много възможни решения.

```
Console.Write("Занплата: ");  
decimal salary;  
while (!decimal.TryParse(Console.ReadLine(), out salary) || salary < 0)  
{  
    Console.Write("Въведете валидна занплата: ");  
}
```

Новият код в случая е `decimal.TryParse(Console.ReadLine(), out salary)`

Ако бяхме поставили тези валидации в set, те щяха да бъдат активирани едва СЛЕД създаването на новия обект Employee чрез Add, а не непосредствено след въвеждането.

```
private static void HighestPaid(List<Employee> employees)  
{  
    Console.WriteLine("\n--- СЛУЖИТЕЛ С НАЙ-ВИСОКА ЗАПЛАТА ---");  
    if (employees.Count > 0)  
    {  
        Вградено property  
        1) Employee maxPaid = employees.MaxBy(e => e.Salary);  
        Console.WriteLine($"Служител с максимална занплата: ");  
        maxPaid.DisplayInfo();  
        предложението на DeepSeek  
        2) Employee highestPaid = employees.DefaultIfEmpty().OrderByDescending(e => e.Salary).First(); //подобна логика в MySQL  
        Console.WriteLine($"Служител с най-висока занплата: {highestPaid}"); //ще работи само за ToString()  
    }  
    else  
    {  
        Console.WriteLine("Няма служители.");  
    }  
}
```

В случая, DeepSeek ми предложи по-бавен вариант, при който се хабят ресурси първо за подреждане в намаляващ ред Descending, а след това се взема първия елемент от него. Тази логика се среща в и MySQL, който ще учите догодина. Когато попитах защо го е предпочел пред MaxBy, отговори, че действително с вграденото свойство е по-ефективно.

MaxBy() - По-ефективен

csharp

```
Employee maxPaid = employees.MaxBy(e => e.Salary);
```

- Едно обхождане на списъка
- Сложност: $O(n)$ - оптимално
- Намира максимума в едно преминаване

OrderByDescending().First() - По-малко ефективен

csharp

```
Employee highestPaid = employees.OrderByDescending(e => e.Salary).First();
```

- Сортиране на целия списък
- Сложност: $O(n \log n)$ в най-добрия случай
- След това взима първия елемент

Ето и много популярен начин на запис на използваните методи един под друг:

```
// Филтриране + проекция в нов обект
var studentNames = students
    .Where(s => s.Age > 25)
    .Select(s => new { s.Name, s.Age })
    .ToList();

// Сортиране
var sortedStudents = students
    .Where(s => s.Age > 25)
    .OrderBy(s => s.Name)
    .ToList();
```

И при търсене

```
Book found = library.Find(b => b.Author.ToLower().Contains("им"));
Book foundStart = library.Find(b => b.Author.StartsWith("Димитър"));
Book foundEnd = library.Find(b => b.Author.EndsWith("Димов"));
```

Последно: все още малко сме използвали тип DateTime / DateOnly. Не забравяйте да четете синтаксиса на от подсказките и разглеждате overloads.

```
static void Main()
{
    List<Grade> journal = new List<Grade>();

    journal.Add(new Grade("Мамемамука", 5.75, new DateTime(2024, 1, 15)));
    journal.Add(new Grade("Бългaрски", 6.00, new DateTime(2024, 1, 16)));
    journal.Add(new Grade("История", 4.50, new DateTime(2024, 1, 17)));
    journal.Add(new Grade("Мамемамука", 5.25, new DateTime()));

    Console.WriteLine("=== УЧЕНИЧЕСКИ ЖУРНАЛ ===");
    foreach (Grade grade in journal)
    {
```

▲ 6 of 18 ▼ **DateTime**(int year, int month, int day)
 Initializes a new instance of the **DateTime** structure to the specified year, month, and day.
year: The year (1 through 9999).

```
static void Main()
{
    List<Grade> journal = new List<Grade>();

    journal.Add(new Grade("Мамемамука", 5.75, new DateTime(2024, 1, 15)));
    journal.Add(new Grade("Бългaрски", 6.00, new DateTime(2024, 1, 16)));
    journal.Add(new Grade("История", 4.50, new DateTime(2024, 1, 17)));
    journal.Add(new Grade("Мамемамука", 5.25, new DateOnly()));

    Console.WriteLine("=== УЧЕНИЧЕСКИ ЖУРНАЛ ===");
    foreach (Grade grade in journal)
    {
```

▲ 2 of 3 ▼ **DateOnly**(int year, int month, int day)
 Creates a new instance of the **DateOnly** structure to the specified year, month, and day.
year: The year (1 through 9999).